



AI Software Engineer Intern

Technical Pre-Assessment

Objective

Build a small **AI-powered document intelligence system** that accepts documents, extracts useful information, answers questions about the document, and provides evidence for its answers.

Candidate Task

You will build a simple web application where a user can:

1. Upload a PDF/TXT document
2. Extract and process the document text
3. Split the text into meaningful chunks
4. Generate embeddings
5. Store embeddings in a vector database
6. Ask questions about the uploaded document
7. Retrieve the most relevant chunks
8. Generate an answer using an LLM
9. Display the answer together with the relevant source chunks

Recommended Stack

You are free to choose your own implementation, but we recommend:

- Backend: Python + FastAPI
- AI / ML: OpenAI API / Gemini API / another LLM API
- Embeddings: OpenAI embeddings / Sentence Transformers
- Vector DB: FAISS / ChromaDB
- Frontend: React / Next.js
- Docker (bonus)

Core Requirements

1. Document Processing

The system should:

- Accept PDF or TXT files
- Extract text
- Clean the extracted text

- Split it into chunks
- Store metadata such as page number / chunk ID

2. Semantic Search

When the user asks a question, for example:

User:
What are the main objectives of this research?

The application should follow this pipeline:

```
Question
↓
Embedding
↓
Vector Search
↓
Top relevant chunks
↓
LLM
↓
Answer + Sources
```

3. Retrieval Augmented Generation (RAG)

You must implement a basic Retrieval Augmented Generation pipeline. The LLM should answer based primarily on the retrieved document context. The system should avoid confidently inventing information that isn't present in the document.

4. Source Attribution

Example expected output:

```
Answer:

The research focuses on improving retinal disease
classification using deep learning models.

Sources:

Page 2
"...the objective of this study is..."

Page 4
"...deep learning-based classification..."
```

This is particularly useful for evaluating whether you understand practical AI systems, rather than simply calling an LLM API.

Bonus Tasks (Optional)

The following are not mandatory but will be viewed favorably if attempted.

Bonus 1: Conversation Memory

Allow the system to understand follow-up questions in context of the previous question, for example:

User: What is the research about?
AI: ...

User: Who proposed this approach?
AI: ...

Bonus 2: Hybrid Search

Combine semantic search with keyword search and compare the results.

Bonus 3: Evaluation

Create 5–10 predefined questions and evaluate retrieval accuracy, answer relevance, hallucination rate, and latency.

Bonus 4: Streaming

Stream the LLM response to the frontend.

Bonus 5: Docker

Provide a docker-compose.yml so the complete application can be started with a single command.

Submission Instructions

- Push your code to a public GitHub repository
- Include a README with setup instructions and a short explanation of your design decisions
- Submit your repository link before the deadline communicated to you separately

Submission Deadline

- **Deadline:** 18 August 2026, 12:00 AM (midnight)
- **Submission:** Please send your GitHub repository link to hr@niduslab.com.